講義 14：INCLUDE 拆檔——TOP / F01 慣例（授課順序：接在講義 20a 之後、講義 9 之前）

> 對應練習：ex14 | 答案程式：`ZR_TR14_CAPSTONE` + `ZR_TR14_TOP` / `ZR_TR14_F01`

本講重點

- INCLUDE 的本質：**原地展開的程式碼片段**，不是獨立可執行的程式
- 實務拆檔慣例：`_TOP`（宣告）、`_F01`（FORM）
- 主程式只剩骨架：一眼看懂整支程式
- **拆檔不是模組化（Modularization）**：好處在可讀性、認得 SAP 慣例、多人同時修改
- 建立、啟用 INCLUDE 的注意事項

1. INCLUDE 的本質

程式一大（正式報表動輒上千行），單一檔案難以維護。`INCLUDE` 陳述式把另一個「INCLUDE 程式」的內容**原地展開**：

```
REPORT zr_tr14_capstone.

INCLUDE zr_tr14_top.         " 展開：全域宣告
...
INCLUDE zr_tr14_f01.         " 展開：FORM 副程式
```

關鍵觀念：

- 編譯時如同把 INCLUDE 檔的文字**貼進**該位置——跟直接寫在主程式裡**完全等價**。拆檔是為了「人」好維護，對程式行為零影響。
- INCLUDE 程式（SE38 建立時類型選 **INCLUDE Program**）**不能單獨執行**，也沒有自己的變數空間——它用的是主程式的環境。
- 同一個 INCLUDE 可以被多支程式共用（本專案就有多支 ZDQM 程式共用 INCLUDE 的例子）——改一處，所有引用它的程式都受影響，改共用 INCLUDE 前先查哪些程式在用（SE38 → 該 INCLUDE → Where-Used）。

1.1 拆檔不是模組化

INCLUDE 常被誤認成「模組化」，其實它只是**整理原始碼檔案**，對照講義 8 / 15 就看得出差別：

	FORM（講義 8）	**Function Module（講義 15）**	**INCLUDE（本講）**
可以被「呼叫」	✓ `PERFORM`	✓ `CALL FUNCTION`	✗ 只會被原地展開
有自己的區域變數	✓	✓	✗ 用主程式的環境
有介面（參數）	✓ `USING` / `CHANGING`	✓ `IMPORTING` / `EXPORTING`…	✗
目的	切出可重複呼叫的邏輯	全系統共用的邏輯	讓大程式好讀好管

那拆檔的好處是什麼？都是「給人看、給團隊用」的，對程式行為沒有任何影響：

1. **大程式好讀、好找**：宣告、事件流程、FORM 各放一處，主程式只剩骨架，雙擊 INCLUDE 名稱就跳過去。
2. **認得 SAP 的慣例**（最重要的一點）：既有客製報表幾乎都拆成 `_TOP` / `_F01`（如本專案 ZDQM 系列）；講義 15 的 Function Group 本身就是系統用 TOP / UXX / U01 這組 include 組成的，Module Pool（Dialog 程式）也是。不學這一講，自己寫程式不受影響，但打開公司既有程式、標準程式會看不懂結構。
3. **多人同時修改**：每個 INCLUDE 是獨立的 Repository 物件、各自鎖定，兩個人可以同時改同一支報表的不同 include。
4. **共用**：多支程式可以引用同一個 include——這是雙面刃，見上一段的 Where-Used 提醒。

所以本講安排在「程式真的變大」之後：講義 13 的總整理已經寫出一支有選擇畫面（Selection Screen）、事件、JOIN、分頁、總頁數回填的完整報表，也讀過實戰程式 ZRFI0004，正是體會拆檔好處的時機。學完可以回頭把講義 13 的作品拆成 `_TOP` / `_F01`。

2. 實務拆檔慣例

檔案	內容	命名
主程式	REPORT、INCLUDE 清單、事件流程（只放 PERFORM）	`ZXXX`
TOP include	TYPES / DATA / CONSTANTS / 選擇畫面	`ZXXX_TOP`
FORM include	所有 FORM 副程式（Subroutine）	`ZXXX_F01`（多了再開 F02、F03…）

本專案實例：`ZDQM0001` + `ZDQM0001F01`。拆完的主程式長這樣（就是全部）：

```
REPORT zr_tr14_capstone.

INCLUDE zr_tr14_top.         " 全域宣告 + 選擇畫面

INITIALIZATION.
  t_b1 = '查詢條件'.

START-OF-SELECTION.
  PERFORM get_data.
  PERFORM display_data.

* FORM 的 INCLUDE 放最後：FORM 定義插在事件中間會造成
* 後續程式碼 "statement not accessible"
INCLUDE zr_tr14_f01.         " FORM 副程式
```

順序規則（因為 INCLUDE 是原地展開，位置就是一切）：

1. `_TOP` 放最前——宣告必須在使用之前。

2. `_F01` 放最後——FORM 區塊之後的可執行碼搆不到，事件會被吃掉。

3. 建立方式（SE38）

1. 先建主程式（Executable Program），寫下 `INCLUDE zr_xxx_top.`。
2. 語法檢查會問「INCLUDE 不存在，要建立嗎？」→ 確認 → 系統帶你建 INCLUDE Program（或雙擊 INCLUDE 名稱直接前進建立）。
3. 把宣告/ FORM 搬進對應 INCLUDE，主程式留骨架。
4. **啟用要一起啟用**：主程式與所有 INCLUDE 都要是 active。INCLUDE 單獨語法檢查/啟用時需要「主程式 context」（系統會問以哪支主程式為準）——共用 INCLUDE 時選錯 context 會看到假錯誤。

> 用 ADT / MCP 工具維護時的對應注意事項（資源路徑、context 啟用）記錄在 `.claude/rules/sap-adt-mcp.md`，跟 SE38 的觀念一致：INCLUDE 永遠依附主程式。

4. 什麼該拆、什麼不該拆

- 幾十行的小程式不用拆——拆檔是為了大程式的可讀性，不是儀式。
- 拆檔的判斷點：宣告區 + FORM 超過一兩百行、或多人同時維護不同部分。
- 共用 INCLUDE 要格外小心：它是「隱形的耦合」，新程式間要共用邏輯，優先考慮 Function Module（講義 15，已學過）或 Class，而不是共用 INCLUDE。

5. 常見錯誤與陷阱

症狀	原因
"statement not accessible"	FORM 的 INCLUDE（或 FORM 區塊）後面還有事件程式碼——F01 移到檔尾
INCLUDE 裡用的變數「未宣告」	INCLUDE 順序錯：宣告（TOP）必須先展開
單獨開 INCLUDE 檢查滿江紅	少了主程式 context——從主程式進入，或檢查時選對主程式
改了共用 INCLUDE，別支程式壞了	先 Where-Used 再動手
INCLUDE 程式按 F8 沒反應	它本來就不可執行，執行主程式

6. 課堂練習

完成 ex14：把一支完整的報表拆成主程式 + `_TOP` + `_F01` 三支，驗證行為與拆檔前完全一致。